講義 27：並行控制與 Lock Object（授課順序：接在講義 9（ALV）之後；延續講義 21 的 Z 資料表）

> 對應練習：ex27 | 答案物件：Lock Object `EZTR21_STUD`（沿用練習 21 的 `ZTR21_STUD`）+ 程式 `ZR_TR27_LOCK_OBJECT`

本講重點

- 為什麼多人同時改同一筆資料會出問題（並行寫入衝突）
- SAP 用什麼機制解決：Enqueue Server / Lock Table（觀念）
- 在 SE11 建立自訂 **Lock Object**（`ENQU` 物件類型）
- 系統自動產生的 `ENQUEUE_*` / `DEQUEUE_*` Function Module——怎麼查、命名規則
- 程式怎麼呼叫這兩個 FM，例外處理該接哪些
- **Lock Mode**（E / S / X / O）的差異與選擇
- **SM12 跟 Lock Object 的關係**：Lock Object 是設計圖，SM12 是查看「目前所有鎖定記錄」的窗口，兩者怎麼對應
- 鎖定什麼情況下會卡住殘留、怎麼用 SM12 排查與清除

1. 為什麼需要 Lock Object：一個會出事的情境

想像 `ZTR21_STUD`（講義 21 建的學生表）有一個維護畫面，兩個承辦人 A、B 同時打開**同一個學號**的資料：

1. A 打開 `S0001`，畫面顯示成績 85
2. B 也打開 `S0001`，畫面同樣顯示 85（B 還不知道 A 正在改）
3. A 把成績改成 90，存檔
4. B 沒看到 A 的改動，把自己畫面上的 85 改成 78，存檔——**B 的存檔直接蓋掉 A 剛存的 90**，A 的異動憑空消失

這叫「**遺失更新（Lost Update）**」，是多人系統的經典並行問題。單機小程式不會遇到，但 SAP 是**多使用者同時上線**的系統，任何有維護畫面的表都可能踩到。

解法不是「程式自己判斷」（例如 A、B 各自 SELECT 一次比對時間戳記，土法煉鋼容易漏case），而是 SAP 提供了一套系統層級的機制：**Lock Object（鎖定物件）**。概念很單純：A 要改之前先跟系統「登記」（ENQUEUE），系統記下「這筆資料現在被 A 佔用」；B 也想改時，系統一查已經有人登記了，直接擋下 B（丟出例外），B 只能等 A 做完、解除登記（DEQUEUE）之後才能繼續。

2. Enqueue Server（負責管理鎖定的服務）與 Lock Table（目前所有鎖定的清單）（觀念，不用深究底層）

SAP 系統有一個獨立的**鎖定管理員（Enqueue Server）**，維護一張**鎖定表（Lock Table）**，記錄「誰、鎖了哪張表的哪一筆、什麼模式」。這張表**不是資料庫表**，是 SAP Kernel 在記憶體裡管理的（可以用 **SM12** 交易碼（Transaction Code）查看目前所有鎖定紀錄）。

關鍵特性：

- 鎖定跟**資料庫交易（COMMIT/ROLLBACK）沒有直接關係**——鎖定要靠程式明確呼叫 DEQUEUE 解除，或整個使用者對話（Session）結束時系統自動清掉殘留的鎖
- 鎖定檢查**只在程式主動呼叫 ENQUEUE 時才生效**，不會自動阻止 Open SQL 的 `UPDATE` / `MODIFY`——這點跟講義 21 學過的「外鍵（Foreign Key）只擋畫面輸入、不擋 Open SQL」是同一種概念：**Lock Object 是一個要靠程式主動配合的機制，不是資料庫層的強制鎖**

3. SE11 建立自訂 Lock Object

1. SE11 → 左邊選 **Lock Object** → 輸入名稱（**系統強制規定要 `E` 開頭**，如 `EZTR21_STUD`，代表 Enqueue——不是單純的命名慣例，打別的字首存檔會直接跳出警告「Start the lock object names with the prefix 'E'」）→ Create
2. **Tables** 頁籤：Primary Table 填要保護的表（`ZTR21_STUD`）
3. **Lock Parameters** 頁籤：勾選要當鎖定鍵的欄位——通常勾整個主鍵（`MANDT` + `ID`），代表「鎖住這一筆特定學生記錄」，不是鎖整張表
4. **Lock Mode**（下一節詳細講）：練習選 `E`（Exclusive/Write Lock，最常用）
5. 存檔、Activate——**啟用的瞬間，系統會自動產生兩個 Function Module**：`ENQUEUE_EZTR21_STUD`、`DEQUEUE_EZTR21_STUD`（命名規則固定是 `ENQUEUE_` / `DEQUEUE_` 接 Lock Object 全名），這兩個 FM 你不用自己寫，SE11 幫你生好

4. 查詢自動產生的 FM

啟用後想找這兩個 FM，兩種方法：

- **SE11 裡**：該 Lock Object 畫面 → Utilities → **Generated Objects**，會列出兩個 FM 名稱與所在的 Function Group
- **SE37 直接查**：`ENQUEUE_EZTR21_STUD` / `DEQUEUE_EZTR21_STUD`，Display 可以看到完整的 Import 參數清單——**參數名稱就是 Lock Parameters 頁籤勾選的欄位名稱（小寫）**，例如本例會有 `MANDT`、`ID` 兩個 Import 參數，各自帶預設值 `SY-MANDT` / 空白

5. Lock Mode：E / S / X / O 的差異

Lock Mode	說明	適用情境
E（Exclusive/Write Lock）	同一時間只能有**一個人**鎖定；同一個使用者可以重複呼叫 `ENQUEUE` 疊加鎖定次數，但要呼叫相同次數的 `DEQUEUE` 才會真正解鎖	**最常用**，本練習選這個——確保同一筆資料同時間只有一人在改
S（Shared/Read Lock）	多人可以**同時**持有 S 鎖（唯讀情境），但只要有人持有 S 鎖，其他人就拿不到 E 鎖	適合「允許多人同時讀取、但寫入要互斥」的情境
X（Exclusive，不可累加）	效果類似 E，但**同一個使用者**重複呼叫 `ENQUEUE` 會直接失敗（不像 E 允許同一人疊加鎖定次數）	適合「絕對只能鎖一次，連自己都不能重入」的嚴格情境
O（Optimistic）	不是真的鎖資料，只記錄狀態供之後比對是否被別人改過（樂觀鎖定）	適合長時間編輯、不想整段時間占著鎖的情境

練習只會用到 `E`，其他三種先有印象即可，遇到實際需求再回頭查。

6. 程式怎麼呼叫：ENQUEUE → 編輯 → DEQUEUE

固定套路三步驟：

```
" 1. 上鎖
CALL FUNCTION 'ENQUEUE_EZTR21_STUD'
  EXPORTING
    mandt          = sy-mandt
    id             = lv_id
  EXCEPTIONS
    foreign_lock   = 1   " 別人已經鎖住了
    system_failure = 2   " 鎖定系統本身出問題（罕見）
    OTHERS         = 3.

IF sy-subrc <> 0.
  " 鎖不到，不能繼續改—通常印訊息告知使用者、中止本次異動
  MESSAGE '資料正被其他人編輯中，請稍後再試' TYPE 'I'.
  RETURN.
ENDIF.

" 2. 真正的編輯邏輯（UPDATE/INSERT/MODIFY...）放這裡

" 3. 解鎖（不管上面編輯成功或失敗，都要記得解鎖，避免鎖定一直占著）
CALL FUNCTION 'DEQUEUE_EZTR21_STUD'
  EXPORTING
    mandt = sy-mandt
    id    = lv_id.
```

三個重點：

1. `ENQUEUE` 一定要接 `EXCEPTIONS foreign_lock` / `system_failure` / `OTHERS`，`sy-subrc <> 0` 就代表鎖不到，**絕對不能無視例外直接往下做編輯**
2. `DEQUEUE` **沒有 EXCEPTIONS 可以接**（它幾乎不會失敗，語法上也不需要）——但一定要呼叫，忘記解鎖會讓這筆資料一直卡住，別人永遠鎖不到，直到你的 Session 結束系統才會自動清掉
3. E 模式下，**同一個使用者**可以對同一筆資料重複呼叫 `ENQUEUE` 而不會被自己擋下來（次數會疊加），但要呼叫**相同次數**的 `DEQUEUE` 才會真正解鎖——這代表「呼叫一次 ENQUEUE 就要對應呼叫一次 DEQUEUE」的規律，程式裡不要漏算

7. SM12 跟 Lock Object 的關係

先講清楚兩者的分工，很多人會誤以為 SM12 是另一套獨立的鎖定機制，其實不是：

- **Lock Object（SE11）是「設計圖」**：定義一個鎖要長什麼樣子——鎖哪張表、用哪些欄位當 Key、Lock Mode 是什麼——並產生 `ENQUEUE_*` / `DEQUEUE_*` 這兩支「按圖施工」用的 FM
- **鎖定表（Lock Table）是「施工結果」**：每次程式呼叫 `ENQUEUE_*` 成功，Enqueue Server 就會在這張表裡**新增一筆記錄**（誰、鎖了哪個 Lock Object、實際的 Key 值是什麼、什麼時間鎖的）；呼叫 `DEQUEUE_*` 就是把那筆記錄**移除**

- **SM12 是「查看施工結果的窗口」**：這個交易碼本身不會鎖任何東西，它只是把 Enqueue Server 記憶體裡目前所有的鎖定記錄（**整個系統，不限於你自己的 Lock Object**）列出來給你看——換句話說，SM12 畫面上每一列，都對應到某個時間點某支程式呼叫某個 Lock Object 的 `ENQUEUE_*` 之後、還沒呼叫對應 `DEQUEUE_*` 的狀態

SM12 畫面欄位跟 Lock Object 的對應關係：

SM12 欄位	對應到
Table Name	Lock Object 的 Primary Table（如 `ZTR21_STUD`）
Lock argument	Lock Parameters 勾選欄位的實際值組合（如 `130S0001`，`130`=Client、`S0001`=學號）
User name	呼叫 `ENQUEUE_*` 的使用者
Time	鎖定建立的時間
Transaction Code / Program	是哪個 T-code / 程式呼叫了 `ENQUEUE_*`

7.1 雙視窗測試：實際看到「被擋下來」

`FOREIGN_LOCK` 這個例外只有在**另一個使用者**（或另一個沒有 DEQUEUE 就結束的 Session）已經持有鎖時才會發生。單一程式跑一次 `programrun` 看不到這個效果（一次執行結束後鎖就釋放了）。實際上課測試方式：

1. 開兩個 SAP GUI 視窗（或兩個不同帳號登入）
2. 視窗 A 執行練習程式，在 `ENQUEUE` 成功、還沒 `DEQUEUE` 之前，用中斷點（Breakpoint）或 `WAIT UP TO 30 SECONDS` 讓程式停在那裡
3. 視窗 B 對**同一個學號**執行同一支程式——應該會在 `ENQUEUE` 那一步收到 `FOREIGN_LOCK`（`sy-subrc = 1`）
4. **SM12** 交易碼：輸入 Table Name 或 User，可以看到目前系統裡所有鎖定紀錄（誰、鎖了哪個 Lock Object、哪些參數值）——視窗 A 還沒 DEQUEUE 前，這裡應該看得到一筆
5. 視窗 A 執行 `DEQUEUE` 或結束程式後，SM12 那筆記錄應該消失，視窗 B 再試一次就會成功

7.2 什麼情況下鎖定會「卡住」，以及怎麼用 SM12 排除

正常情況下鎖定都是程式自己成對呼叫 `ENQUEUE` / `DEQUEUE` 來管理，不需要人工介入。但以下情況會讓鎖**沒有機會被正常 `DEQUEUE`**，變成殘留在鎖定表裡：

- 程式在 `ENQUEUE` 成功之後、`DEQUEUE` 之前發生 Dump（未預期的執行期錯誤（Runtime Error））
- 使用者在維護畫面卡住不動、放著不管，或直接關掉 SAP GUI 視窗（沒有走正常的「離開/取消」流程）
- **無頭測試工具呼叫到需要畫面互動的功能，卡住等待、最終被強制斷線**（例如 `programrun` 呼叫到 `VIEW_MAINTENANCE_CALL` 這種會開整個維護畫面的呼叫，`programrun` 沒有畫面可以操作，最終連線逾時中斷——這種「非正常結束」的呼叫路徑，理論上也不會執行到後面的 `DEQUEUE` 那一行）

上述任何一種情況發生後，該怎麼確認鎖有沒有真的卡住：

1. **SM12** 輸入 Table Name（或直接輸入自己的 User name）→ Enter，看看有沒有殘留的那一筆
2. **多數情況下不用擔心**：SAP Kernel 對「Session / Mode 已經結束、但鎖還沒明確釋放」這種狀況，通常會在 Session 真正終止時自動一併清掉相關的鎖，只是可能有短暫延遲，不是立即消失

3. 如果過一段時間再查 SM12 那筆記錄還在（真的卡住了），可以在 SM12 選取那一列 → **Delete Locks**（垃圾桶圖示或選單 Lock Entry → Delete）手動刪除——**這是維運層級的操作，要先確認清楚這個鎖真的沒人在用了才刪，否則等於是強行打斷一個還在進行中的使用者作業**

> 練習 27 / 28 的情境都不需要真的用到「手動刪除」這個功能，但要知道 SM12 是排查「Lock Object 好像卡住了」這類問題時**第一個要查的地方**——遇到 `FOREIGN_LOCK` 但想不通是誰鎖住的、或懷疑有殘留鎖，先去 SM12 看一眼，比瞎猜有效率得多。